



兰州工业学院

课 设 报 告

(2024—2025 学年第一学期)

课设名称 电子系统设计--温室光照控制系统设计

学 院 电子信息工程学院

专业班级 专升本电信 23 班

姓 名 姜 勇

学 号 ZB2305010116

指导教师 陶冶、朱红娟

起止日期 2024. 11. 25—2024. 11. 29

目录

1. 需求分析与方案论证.....	1
1.1 需求分析.....	1
1.2 方案论证.....	1
2. 单元电路设计.....	2
2.1 电路设计目标.....	2
2.2 电路设计过程.....	2
2.3 电路原理图绘制.....	3
2.3.1 单片机最小系统电路原理图.....	3
2.3.2 LCD1602 显示电路原理图.....	3
2.3.3 按键电路原理图.....	3
2.3.4 可调滑动变阻器（模拟光通量）电路原理图.....	4
2.3.5 步进电机与驱动电路原理图.....	4
2.3.6 串口通信电路原理图.....	4
2.3.7 总体电路设计.....	5
3. 微处理器最小系统设计.....	5
3.1 微处理器选择与最小系统设计.....	5
3.2 最小系统测试.....	5
4. 主程序设计.....	5
4.1 程序设计概要.....	5
4.2 主程序架构与流程图.....	6
4.2.1 系统初始化模块.....	6
4.2.2 光照采集模块.....	6
4.2.3 显示模块.....	7
4.2.4 按键模块.....	8
4.2.5 步进电机控制模块.....	8
4.2.6 串口通信模块.....	9
4.3 完整主程序流程图.....	9

5. 采集程序设计.....	10
5.1 采集功能需求.....	10
5.2 采集模块设计.....	10
5.3 代码实现.....	10
6. 显示程序设计.....	12
6.1 显示功能需求.....	12
6.2 显示模块设计.....	12
6.3 代码实现.....	12
7. 控制程序设计.....	16
7.1 控制功能需求.....	16
7.2 控制模块设计.....	16
7.3 代码实现.....	16
8. 步进电机控制设计.....	20
8.1 控制功能需求.....	20
8.2 控制模块设计.....	20
8.3 代码实现.....	21
9. 系统实现与运行结果.....	25
9.1 系统实现.....	25
9.2 运行结果说明.....	26
9.3 问题与解决方案.....	26
课设总结.....	28

课设时间	2024. 11. 25—2024. 11. 29
课设地点	教室
课设目的 让学生通过温室光照控制系统的设计，掌握方案论证、单元电路设计、电路原理图绘制、微处理器最小系统设计等关键技能，并能够实现光通量值的采集显示、光通量标准范围的设置以及光线异常的控制。同时，培养学生的团队合作能力、沟通协调能力，并综合运用所学知识形成设计方案，最终通过撰写设计报告和答辩来展示课程设计的成果。	
课设 日志	
日期：2024. 11. 25 课设内容： 1. 需求分析与方案论证 1.1 需求分析 本次课程设计目标是设计一个基于微处理器的控制系统，主要包括数据采集、显示、控制等功能，同时通过串口模拟蓝牙进行通信。系统要求具备以下功能： • 数据采集：通过可调滑动变阻器模拟光通量变化。 • 数据显示：将采集到的数据通过 LCD1602 液晶屏实时显示。 • 控制功能：使用按键设置光通量的上下限范围。当光通量超过设置的上下限时，控制步进电机打开或关闭遮光板。 • 串口通信：通过 UART 串口通信实现数据交换，可以与其他设备进行通信。 • 步进电机控制遮光板：根据光通量的值，通过步进电机控制遮光板的转动，右转为打开，反转为关闭。 1.2 方案论证 根据需求分析，提出多个可行方案，并进行论证： 方案一：使用硬件蓝牙模块（如 HC-05）进行无线通信。	

- 优点：蓝牙模块稳定，易于连接。
- 缺点：增加硬件成本。

方案二：通过串口模块模拟蓝牙功能，利用单片机的串口进行数据传输。

- 优点：成本较低，系统集成度高。
- 缺点：通信距离有限，需要手动模拟蓝牙协议。

方案三：通过按键设置光通量上下限，通过变阻器模拟光通量的变化；当光通量超出上下限时，控制步进电机的转动，调节遮光板的开合状态。

- 优点：能够根据光通量的变化调节遮光板的开合，满足自动化调节需求。
- 缺点：按键操作和范围设置的用户体验可能需要进一步优化。

最终选择方案二和三，使用串口模块模拟蓝牙通信，并通过按键控制光通量范围及步进电机控制遮光板的开关。

日期：2024. 11. 26

课设内容：

2. 单元电路设计

2.1 电路设计目标

设计一个基于 C51 单片机的温室光照控制系统，实现数据采集、显示、控制、串口通信和步进电机控制遮光板的功能。

2.2 电路设计过程

- **设计方法：**使用 C51 单片机进行系统控制，通过串口模块实现模拟蓝牙通信，使用步进电机驱动电路控制步进电机的转动，从而调节遮光板的位置。
- **元器件选择：**
 - **单片机：**C51 单片机（如 AT89C51），作为主控制器。
 - **可调滑动变阻器：**模拟光通量变化。
 - **显示模块：**LCD1602 液晶屏用于显示光通量和设定范围。
 - **按键：**用于设置光通量的上下限。
 - **串口模块：**使用内置 usart 模块进行通信。
 - **步进电机：**使用 28BYJ-48 步进电机，配合 ULN2003 驱动模块进行控制。

2.3 电路原理图绘制

2.3.1 单片机最小系统电路原理图

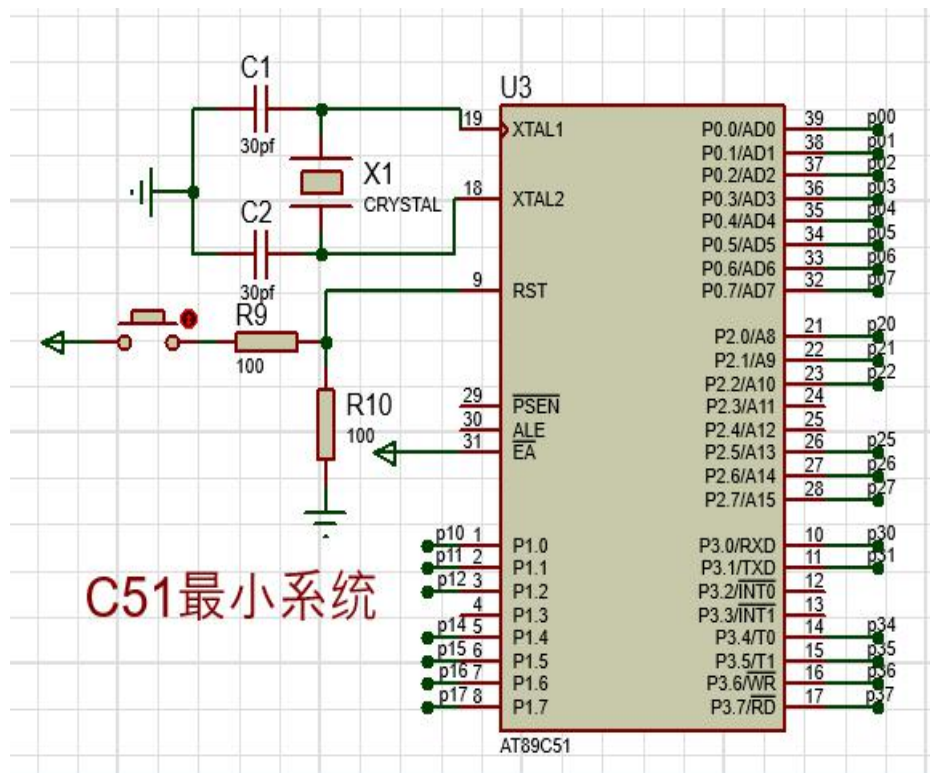


图 2-1 最小系统电路

2.3.2 LCD1602 显示电路原理图

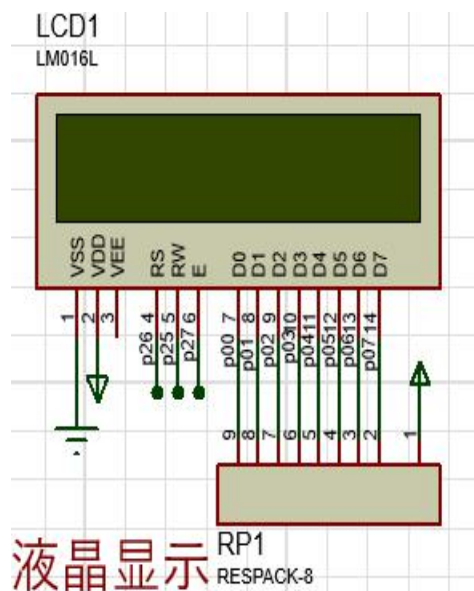


图 2-2 显示电路

2.3.3 按键电路原理图



图 2-3 按键电路

2.3.4 可调滑动变阻器（模拟光通量）电路原理图

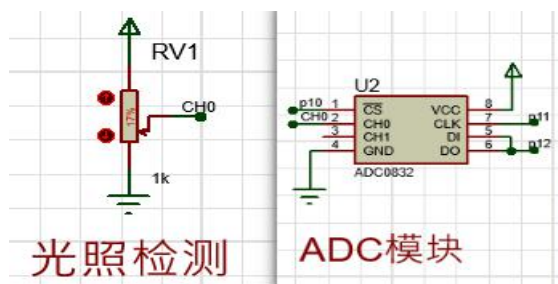


图 2-4 光照采集模拟电路

2.3.5 步进电机与驱动电路原理图

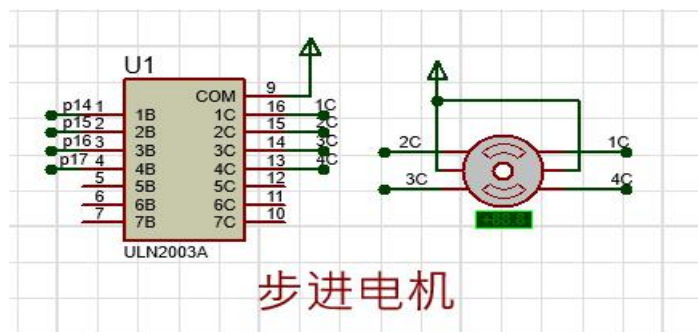


图 2-5 步进电机驱动电路

2.3.6 串口通信电路原理图

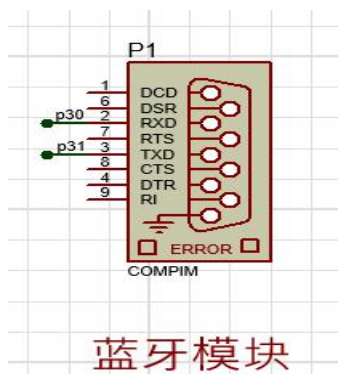


图 2-6 串口电路

2.3.7 总体电路设计

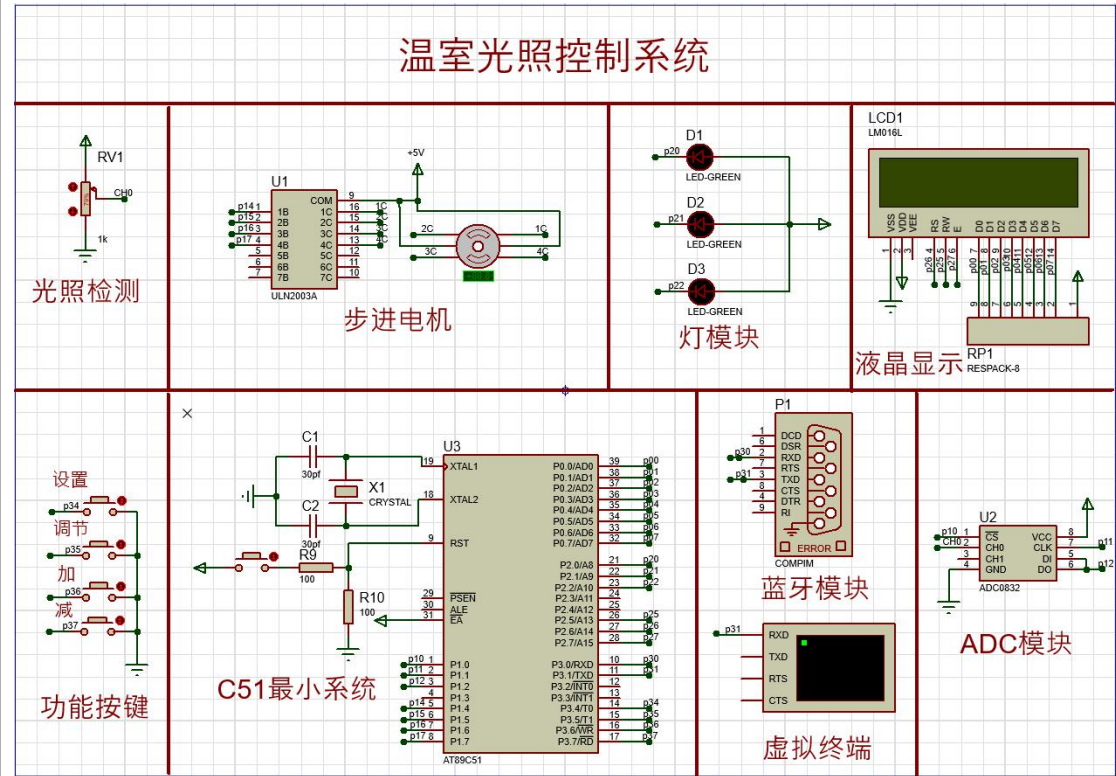


图 2-7 总体电路

3. 微处理器最小系统设计

3.1 微处理器选择与最小系统设计

- **微处理器选择：**本设计选用 AT89C51 单片机，具备足够的 I/O 接口和处
理能力，适用于本项目需求。
- **最小系统设计：**包括电源、复位电路等，同时加入串口和步进电机控制
电路，确保系统能够稳定运行。

3.2 最小系统测试

通过连接电源并调试程序，测试最小系统是否能够正常启动，检查各个模块
的工作情况，如串口通信、数据采集、按键输入、步进电机控制等。

日期：2024. 11. 27

课设内容：

4. 主程序设计

4.1 程序设计概要

主程序包括以下几个核心功能模块：

1. **系统初始化：**初始化串口、LCD、传感器、按键、步进电机等硬件。

2. **数据采集**: 从可调滑动变阻器采集光通量数据。
3. **显示功能**: 将当前光通量数据及上下限范围显示在 LCD1602 屏幕上。
4. **按键输入**: 通过按键输入设置光通量的上下限。
5. **步进电机控制**: 根据采集到的光通量, 判断是否需要控制步进电机来调节遮光板开合。
6. **串口通信**: 通过 UART 串口发送数据到外部设备。

4.2 主程序架构与流程图

4.2.1 系统初始化模块



图 4-1 初始化流程图

4.2.2 光照采集模块



图 4-2 光照采集流程图

4.2.3 显示模块



图 4-3 显示流程图

4.2.4 按键模块

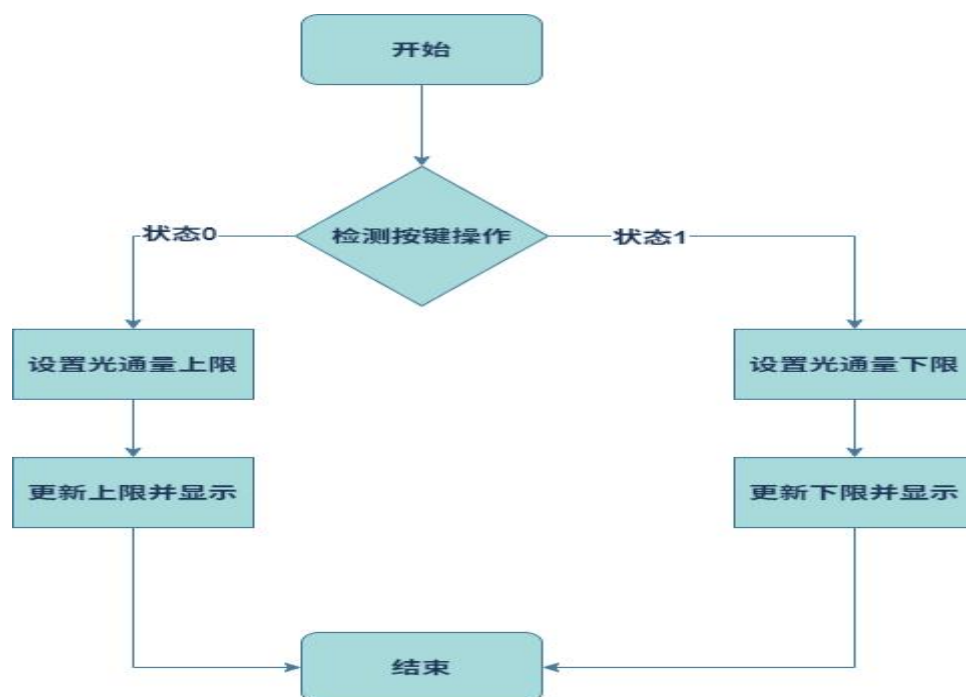


图 4-4 按键流程图

4.2.5 步进电机控制模块

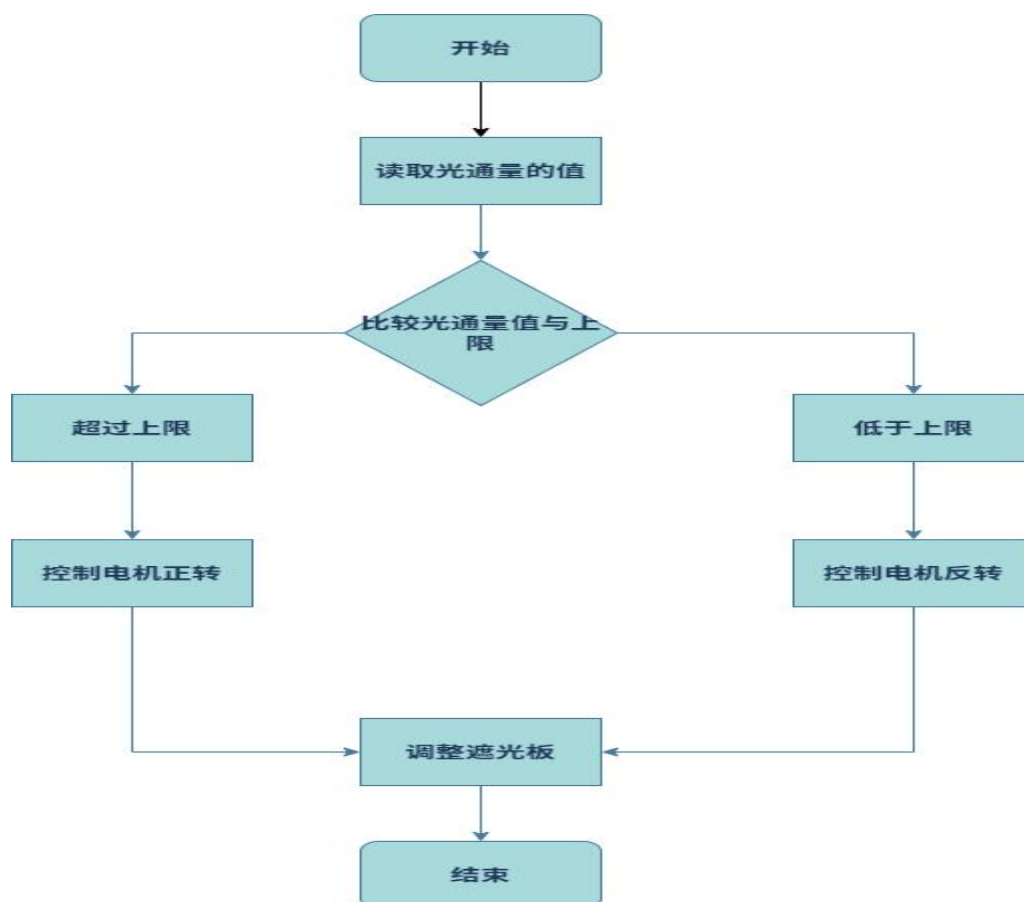


图 4-5 模拟遮光板流程图

4.2.6 串口通信模块



图 4-6 串口通信流程图

4.3 完整主程序流程图

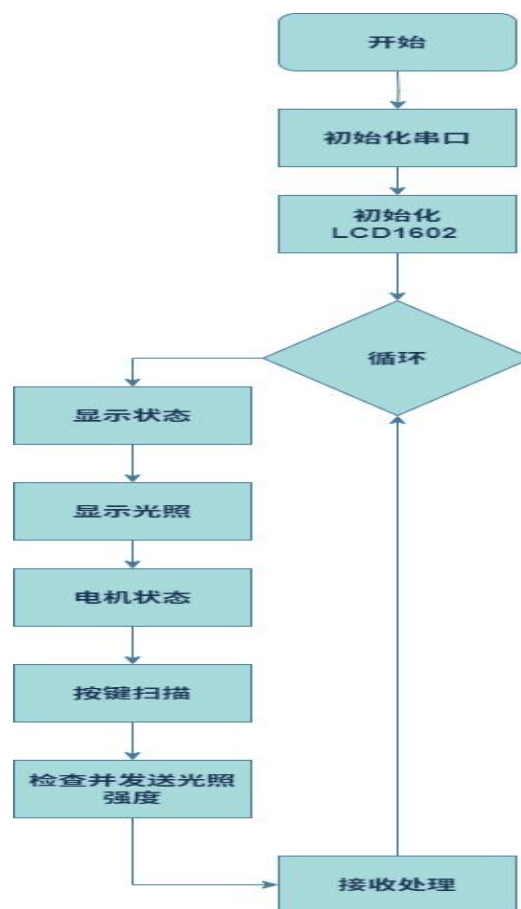


图 4-7 主流程图

5. 采集程序设计

5.1 采集功能需求

- 数据采集：通过可滑动变阻器模拟光通量数据。

5.2 采集模块设计

使用 ADC 接口读取变阻器的模拟值，转换为光通量数据。

5.3 代码实现

```
#include "adc0832.h"

u8 ADC_read_data(bit channel)
{
    u8 i, dat0 = 0, dat1 = 0; // 初始化数据变量
    cs = 0;                    // CS (Chip Select) 置低，选
    择 ADC0832 芯片
    clk = 0;                    // CLK (Clock) 置低
    dio = 1;                    // DIO (Data In/Out) 置高
    delay(2);                   // 短暂延时
    clk = 1;                    // CLK 上升沿
    delay(2);
    clk = 0;                    // CLK 下降沿
    dio = 1;                    // DIO 再次置高
    delay(2);
    clk = 1;                    // CLK 上升沿
    delay(2);
    clk = 0;                    // CLK 下降沿
    dio = channel;              // 设置 DIO 为 channel 值来选择输
    入通道
    delay(2);
    clk = 1;                    // CLK 上升沿
```

```

    delay(2);

    clk = 0;                                // CLK 下降沿
    dio = 1;                                // DIO 置高
    delay(2);

    // 开始读取高位字节
    for (i = 0; i < 8; i++)
    {
        clk = 1;                            // CLK 上升沿
        delay(2);
        clk = 0;                            // CLK 下降沿
        delay(2);
        dat0 = dat0 << 1 | dio; // 将 DIO 状态左移一位，并将当前 DIO
状态加入到 dat0
    }

    // 继续读取低位字节
    for (i = 0; i < 8; i++)
    {
        dat1 = dat1 | ((u8)(dio) << i); // 将 DIO 状态按位或入 dat1
        clk = 1;                        // CLK 上升沿
        delay(2);
        clk = 0;                        // CLK 下降沿
        delay(2);
    }

    cs = 1;                                // CS 置高，结
束通信

    return (dat0 == dat1) ? dat0 : 0;    // 如果两次读取的数据相同，
则返回该数据；否则，可能发生了错误，返回 0

```

```
}
```

6. 显示程序设计

6.1 显示功能需求

- 显示光通量机上下限：LCD1602 显示当前采集的光通量数据以及按键设定的上下限

6.2 显示模块设计

使用 16X2LCD 模块，通过并口或 I2C 接口与单片机连接，将光通量数据以及上下限范围实时显示。

6.3 代码实现

日期：2024.11.28

课设内容：

```
#include "lcd1602.h"

// 定义两个缓冲区，用于存储要显示在 LCD 上的字符串
char LED_buffer1[16] = "";
char LED_buffer2[16] = "lux:          lx";

// 光照强度相关的变量
unsigned short adc = 0;  // ADC 读数
float lux = 0.0;  // 计算出的光照强度

// 向 LCD 写入命令
void lcd1602_write_cmd(u8 cmd)
{
    RS = 0;  // 设置 RS 为 0 表示写入的是命令
    RW = 0;  // 设置 RW 为 0 表示写操作
    EN = 0;  // 禁用使能信号
    P0 = cmd;  // 将命令写入 P0 端口
    delay(1);  // 延时
    EN = 1;  // 使能信号高电平触发
    delay(1);  // 延时
```

```

        EN = 0;    // 使能信号低电平结束
    }
    // 向 LCD 写入数据
    void lcd1602_write_dat(u8 dat)
    {
        RS = 1;    // 设置 RS 为 1 表示写入的是数据
        RW = 0;    // 设置 RW 为 0 表示写操作
        EN = 0;    // 禁用使能信号
        P0 = dat;   // 将数据写入 P0 端口
        delay(1);   // 延时
        EN = 1;    // 使能信号高电平触发
        delay(1);   // 延时
        EN = 0;    // 使能信号低电平结束
    }
    // 初始化 LCD
    void lcd1602_init()
    {
        lcd1602_write_cmd(0x38);    // 设置 8 位数据接口，两行显示，5x7 点
        阵
        lcd1602_write_cmd(0x0c);    // 显示开，光标关，不闪烁
        lcd1602_write_cmd(0x06);    // 文字不动，地址自动加 1
        lcd1602_write_cmd(0x01);    // 清屏
    }
    // 清除 LCD 显示
    void lcd1602_clear()
    {
        lcd1602_write_cmd(0x01);    // 清屏命令
    }

```



```

}

// 在指定位置显示字符串
void lcd1602_show_string(u8 x, u8 y, char str[16])
{
    u16 i;
    if (y) x |= 0x40;    // 如果 y=1，则设置第二行
    x |= 0x80;    // 设置起始地址
    lcd1602_write_cmd(x);    // 写入地址
    for (i = 0; str[i] != '\0' && i < 16; i++) {
        lcd1602_write_dat(str[i]);    // 逐字符写入数据
    }
}

// 显示当前状态
void lcd1602_show_state()
{
    // 从 ADC 读取数据并计算光照强度
    adc = ADC_read_data(0);    // 读取通道 0 的数据
    adc = adc * 16;    // 根据 ADC 特性调整数值
    lux = (1 - ((double)adc / 4095.0)) * 1000.0;    // 计算光照强度
    // 根据光照强度更新显示内容和 LED 状态
    if (lux < lower_limit) {    // 如果光照强度低于下限
        LED_buffer1[0] = 'd';
        LED_buffer1[1] = 'a';
        LED_buffer1[2] = 'r';
        LED_buffer1[3] = 'k';
        LED_buffer1[4] = ' ';
    }
}

```

```

        LED_buffer1[5] = ' ';

        led1 = 0;    // 设置 LED1 状态
        led2 = 1;    // 设置 LED2 状态
        led3 = 1;    // 设置 LED3 状态

    } else if (lux >= lower_limit && lux < upper_limit) {    // 如果
光照强度在上下限之间

        LED_buffer1[0] = 'n';
        LED_buffer1[1] = 'o';
        LED_buffer1[2] = 'r';
        LED_buffer1[3] = 'm';
        LED_buffer1[4] = 'a';
        LED_buffer1[5] = 'l';

        led1 = 1;    // 设置 LED1 状态
        led2 = 0;    // 设置 LED2 状态
        led3 = 1;    // 设置 LED3 状态
    } else {    // 如果光照强度高于上限

        LED_buffer1[0] = 's';
        LED_buffer1[1] = 't';
        LED_buffer1[2] = 'r';
        LED_buffer1[3] = 'o';
        LED_buffer1[4] = 'n';
        LED_buffer1[5] = 'g';

        led1 = 1;    // 设置 LED1 状态
        led2 = 1;    // 设置 LED2 状态
        led3 = 0;    // 设置 LED3 状态
    }

    // 在 LCD 的第一行显示状态

```

```

        lcd1602_show_string(0, 0, LED_buffer1);
    }

```

7. 控制程序设计

7.1 控制功能需求

- 按键控制上下限：使用按键输入设置光通量的上下限范围。

7.2 控制模块设计

通过读取按键输入来调整上下限的值，设定光通量的触发范围。

7.3 代码实现

```

#include "key.h"

// 定义显示温度上限和下限的字符串
u8 show_temp_upper[16] = {"upper:      "};
u8 show_temp_lower[16] = {"lower:      "};

// 模式标志：0 为正常模式，1 为设置模式
u8 mode = 0;

// 设置标志：在设置模式中，0 表示设置上限，1 表示设置下限
u8 setting = 0;

// 闪烁计数器
u8 blink_counter = 0;

// 温度上限和下限值
u16 upper_limit = 700;    // 上限默认值
u16 lower_limit = 300;    // 下限默认值

// 按键扫描函数
void key_scan()
{
    // K1 键按下切换模式
    if (!K1) {    // 如果 K1 被按下
        delay(200);    // 延时去抖动
    }
}

```

```

        if (!K1) {    // 再次检查确认 K1 是否真的被按下

            mode = !mode;    // 切换模式

            setting = 0;    // 重置设置状态

            while (!K1);    // 等待按键释放

        }

    }

    // 在设置模式下处理其他按键

    if (mode == 1) {

        // K2 键按下切换设置目标

        if (!K2) {

            delay(200);    // 去抖动

            if (!K2) {

                setting = !setting;    // 切换设置目标

                while (!K2);    // 等待按键释放

            }

        }

        // 根据当前设置目标调整上下限

        if (setting == 0) {    // 设置上限

            if (!K3) {    // K3 增加上限

                delay(200);

                if (!K3) {

                    upper_limit += 10;    // 增加上限

                }

            }

            if (!K4) {    // K4 减少上限

                delay(200);

                if (!K4) {

```

```

        upper_limit -= 10;    // 减少上限
    }

}

} else {    // 设置下限
    if (!K3) {    // K3 增加下限
        delay(200);
        if (!K3) {
            lower_limit += 10;    // 增加下限
        }
    }

    if (!K4) {    // K4 减少下限
        delay(200);
        if (!K4) {
            lower_limit -= 10;    // 减少下限
        }
    }

}

}

}

}

// 显示温度函数
void show_temp()
{
    // 正常模式下显示光照强度
    if (mode == 0) {
        LED_buffer2[6] = (int)lux % 1000 / 100 + '0';    // 百位
        LED_buffer2[7] = (int)lux % 1000 % 100 / 10 + '0';    // 十
位

```

```

        LED_buffer2[8] = (int)lux % 1000 % 100 % 10 + '0';    // 个
位

        LED_buffer2[9] = '.';    // 小数点

        LED_buffer2[10] = (int)(lux * 10) % 1000 % 100 % 10 + '0';    //
小数点后一位

        lcd1602_show_string(0, 1, LED_buffer2);    // 在 LCD 上显示
光照强度

    } else {    // 设置模式下显示温度上下限

        // 更新显示的温度上限字符串

        show_temp_upper[7] = upper_limit % 1000 / 100 + '0';
        show_temp_upper[8] = upper_limit % 1000 % 100 / 10 + '0';
        show_temp_upper[9] = upper_limit % 1000 % 100 % 10 + '0';
        show_temp_upper[10] = ' ';
        show_temp_upper[11] = ' ';
        show_temp_upper[12] = ' ';

        // 更新显示的温度下限字符串

        show_temp_lower[7] = lower_limit % 1000 / 100 + '0';
        show_temp_lower[8] = lower_limit % 1000 % 100 / 10 + '0';
        show_temp_lower[9] = lower_limit % 1000 % 100 % 10 + '0';
        show_temp_lower[10] = ' ';
        show_temp_lower[11] = ' ';
        show_temp_lower[12] = ' ';
        show_temp_lower[13] = ' ';

        // 通过闪烁计数器实现闪烁效果

        if (blink_counter % 2 == 0) {

            if (setting == 0) {    // 当前设置的是上限

                show_temp_upper[7] = ' ';    // 使上限值闪烁

```

```

        show_temp_upper[8] = ' ';
        show_temp_upper[9] = ' ';
    } else {    // 当前设置的是下限
        show_temp_lower[7] = ' ';    // 使下限值闪烁
        show_temp_lower[8] = ' ';
        show_temp_lower[9] = ' ';
    }
}

// 在 LCD 上显示温度上下限
lcd1602_show_string(0, 0, show_temp_upper);
lcd1602_show_string(0, 1, show_temp_lower);
// 更新闪烁计数器
blink_counter++;
if (blink_counter >= 10) {
    blink_counter = 0;    // 重置闪烁计数器
}
}
}

```

8. 步进电机控制设计

8.1 控制功能需求

- 步进电机控制遮光板的开合：根据光通量是否超过或低于上限，控制步进电机转动。
- 右转：当光通量超过上限时，步进电机右转，打开遮光板。
- 反转：当光通量低于上限时，步进电机反转，关闭遮光板。

8.2 控制模块设计

步进电机使用 28BYJ-48 型号电机，配合 ULN2003 驱动模块控制，每次控制步进电机进行一定角度的转动来模拟遮光板的开合。

8.3 代码实现

```
#include "motor.h"

#define STOP_STEPS 5    // 停止步数阈值

// 定义全局变量

u8 dir = 0;    // 电机转动方向（0 为正向，1 为反向）

u8 step = 0;    // 当前步数

u8 motor_running = 1;    // 电机是否正在运行

u16 motor_steps = 0;    // 记录电机已经走过的步数


u8 manual_control = 0;    // 手动控制标志

int lux_triggered = 0;    // 光照强度触发标志

u8 add[16]={" "};

// 发送步进电机脉冲

void step_motor_28BYJ48_send_pulse(u8 step, u8 dir)
{
    u8 temp = step;
    if (dir == 0) temp = 7 - step;    // 如果方向为 0，则反转步序
    switch (temp)
    {
        case 0:
            IN1_A = 1; IN2_B = 0; IN3_C = 0; IN4_D = 0;
            break;
        case 1:
            IN1_A = 1; IN2_B = 1; IN3_C = 0; IN4_D = 0;
            break;
        case 2:
            IN1_A = 0; IN2_B = 1; IN3_C = 0; IN4_D = 0;
```



```

        break;

    case 3:
        IN1_A = 0; IN2_B = 1; IN3_C = 1; IN4_D = 0;
        break;

    case 4:
        IN1_A = 0; IN2_B = 0; IN3_C = 1; IN4_D = 0;
        break;

    case 5:
        IN1_A = 0; IN2_B = 0; IN3_C = 1; IN4_D = 1;
        break;

    case 6:
        IN1_A = 0; IN2_B = 0; IN3_C = 0; IN4_D = 1;
        break;

    case 7:
        IN1_A = 1; IN2_B = 0; IN3_C = 0; IN4_D = 1;
        break;

    default:
        IN1_A = 0; IN2_B = 0; IN3_C = 0; IN4_D = 0;
        break;
}

}

// 控制电机状态
void motor_state()
{
    int i;

    if (!manual_control) {    // 如果不是手动控制
        if (motor_running) {    // 如果电机正在运行

```

```

        if (lux > upper_limit) {    // 如果光照强度大于上限
            step_motor_28BYJ48_send_pulse(step++,
dir);    // 正向转动一步

            add[0] = '0';    // 更新 LED 缓冲区
        } else if (lux < upper_limit) {    // 如果光照强度小
于上限

            step_motor_28BYJ48_send_pulse(step++, !dir);

            // 反向转动一步

            add[0] = '0';    // 更新 LED 缓冲区
        }

        if (step == 8) step = 0;    // 如果步数达到 8，重置步
数

        delay(1);    // 延时

        motor_steps++;    // 累加步数

        if (motor_steps >= STOP_STEPS) {    // 如果步数达到停
止步数

            motor_running = 0;    // 停止电机

            if (lux > upper_limit) {

                add[0] = 'K';    // 更新 LED 缓冲区

            } else {

                add[0] = 'D';    // 更新 LED 缓冲区

            }

            lux_triggered = (lux > upper_limit) ? 1 :
-1;    // 设置光照强度触发标志

        }

    } else {    // 如果电机未运行

        if ((lux > upper_limit && lux_triggered != 1) || (lux

```

```

< upper_limit && lux_triggered != -1)) {

    motor_running = 1;    // 重新启动电机

    motor_steps = 0;    // 重置步数

    lux_triggered = 0;    // 重置光照强度触发标志

}

}

}

lcd1602_write_cmd(0x8d);    // 设置 LCD 显示位置
for (i = 0; i < 16; i++) {

    lcd1602_write_dat(add[i]);    // 显示 LED 缓冲区内容

}

}

// 模拟蓝牙接收字符并控制电机
void simulate_bluetooth(char received_char)
{

    int i;

    manual_control = 1;    // 设置为手动控制

    if (received_char == 'a' || received_char == 'd') {    // 如果接收到'a'或'd'

        if (motor_running == 0) {    // 如果电机未运行

            motor_running = 1;    // 启动电机

            motor_steps = 0;    // 重置步数

        }

        if (received_char == 'a') {    // 如果接收到'a'

            step_motor_28BYJ48_send_pulse(step++, dir);    // 正向转动一步

            add[0] = '0';    // 更新 LED 缓冲区

```

```

        } else if (received_char == 'd') {    // 如果接收到'd'
            step_motor_28BYJ48_send_pulse(step++, !dir);    // 反
向转动一步

            add[0] = '0';    // 更新 LED 缓冲区
        }

        if (step == 8) step = 0;    // 如果步数达到 8，重置步数
        delay(1);    // 延时
        motor_steps++;    // 累加步数
        if (motor_steps >= STOP_STEPS) {    // 如果步数达到停止步数
            motor_running = 0;    // 停止电机
            if (received_char == 'a') {
                add[0] = 'K';    // 更新 LED 缓冲区
            } else {
                add[0] = 'D';    // 更新 LED 缓冲区
            }
        }
    }

    lcd1602_write_cmd(0x8d);    // 设置 LCD 显示位置
    for (i = 0; i < 16; i++) {
        lcd1602_write_dat(add[i]);    // 显示 LED 缓冲区内容
    }
}

```

9. 系统实现与运行结果

9.1 系统实现

硬件部分：包括 C51 单片机、可滑动变阻器、LCD1602 液晶屏、步进电机驱动模块、按键输入和步进电机。

软件部分：包括数据采集、显示、控制、步进电机控制等功能模块。

9.2 运行结果说明

- 功能验证：系统能够通过可调滑动变阻器实时采集光通量数据，并在LCD1602上显示；通过按键设置光通量的上下限，并通过步进电机控制遮光板的开合。
- 性能测试：系统响应速度较快，步进电机能够平滑调节遮光板的开关，控制准确。

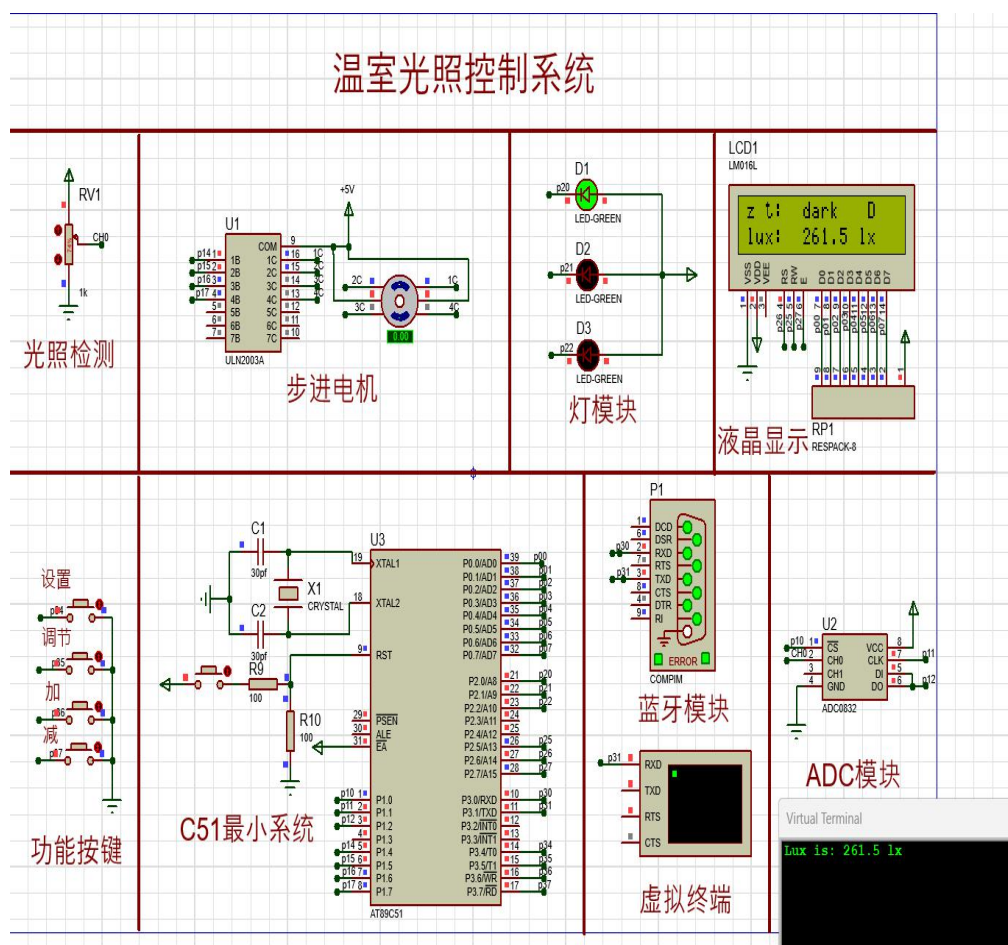


图 9-1 运行结果

9.3 问题与解决方案

- 问题：按键设置的光通量上下限范围时，可能出现按键抖动导致的误操作。
- 解决方案：在按键输入处理中加入去抖动延时，确保按键稳定触发。

日期：2024. 11. 29

课设内容：

答辩提问：

1. LCD1602 液晶屏是由单片机的哪些引脚控制的？

RS：寄存器选择引脚，连接到单片机的 P2.5。

RW：读/写选择引脚，连接到单片机的 P2.6，通常在写操作时接低电平。

E：使能引脚，连接到单片机的 P2.7，用于触发数据的读写。

D0-D7：8 位数据总线，连接到单片机的 P0 口（P0.0-P0.7）

2. 在 Proteus 仿真中网络标识是怎么用的？

标识连接点：网络标识可以用于标识电路中的特定连接点，这样在电路的其他地方可以通过相同的网络标识来引用这个连接点。

跨页面连接：在大型项目中，如果电路图被分割成多个页面，网络标识可以用于在不同页面之间建立电气连接。

自动化布线：在 Proteus 中，网络标识可以用于自动化布线工具，以确保所有具有相同网络标识的点都被正确连接。

子电路设计：在设计子电路或模块时，网络标识可以用于定义模块的输入和输出端口。

3. 串口模拟蓝牙模块是干嘛的？

模拟蓝牙模块发送和接收数据的行为，使得开发者能够测试和调试他们的应用程序。在实际应用中，当蓝牙模块接收到特定的指令（如'a'或'd'），它可以通过串口发送这些指令到微控制器，然后微控制器根据接收到的指令来控制电机或其他设备。

课设总结

在本次课程设计中，我们成功完成了一个基于微处理器的温室光照控制系统设计。通过这个项目，我们掌握了方案论证、单元电路设计、电路原理图绘制、微处理器最小系统设计等关键技能，并且实现了光通量值的采集显示、光通量标准范围的设置以及光线异常的控制。同时，我们也培养了团队合作能力、沟通协调能力，并综合运用所学知识形成设计方案，最终通过撰写设计报告和答辩来展示课程设计的成果。

在需求分析与方案论证阶段，我们明确了系统的基本功能需求，并提出了多个可行的方案进行比较，最终选择了使用串口模块模拟蓝牙通信，并通过按键控制光通量范围及步进电机控制遮光板的开关的方案。在单元电路设计阶段，我们设计了基于 C51 单片机的系统，实现了数据采集、显示、控制、串口通信和步进电机控制遮光板的功能。电路原理图的绘制和微处理器最小系统的设计，确保了系统能够稳定运行。

在程序设计部分，我们完成了主程序设计、采集程序设计、显示程序设计、控制程序设计以及步进电机控制设计。这些程序模块协同工作，实现了系统的核心功能。通过实际的硬件搭建和软件编程，我们验证了系统的功能，并进行了性能测试。测试结果表明，系统响应速度快，步进电机能够平滑调节遮光板的开关，控制准确。

总的来说，这次课程设计不仅让我们将理论知识应用到实践中，还提升了我们解决实际问题的能力。我们遇到的问题，如按键抖动导致的误操作，通过加入去抖动延时得到了有效解决。整个项目的经历对我们的工程实践能力和创新思维能力的培养都有着积极的影响。

课设成绩评定表

课设名称		温室光照控制系统设计	
序号	考核项目	分值分布	成绩
1	出勤表现	15	
2	完成任务情况	25	
3	答辩情况	30	
4	课设报告	30	
总分			
指导教师签字			

备注：考核项目及分值分布应依据课设大纲和课设内容确定。归档时应归入成绩评定原始记录。